CS4052 Computación Paralela y Distribuida – 2026-II

Proyecto de Laboratorio – **Entrega final**

Caso 5: KNN en paralelo con MPI (mpi4py)

Clasificación de dígitos (*load_digits*)

Profesor: José Fiestas – Grupo 7

Integrante	**Participación**
Barzola Estrella, Johar Jared	100 %
Huaman Casma, Leonardo Fabian	100 %
Izaguirre Zavaleta, Luis Fernando	100 %

(100 % indica que el integrante contribuyó en forma equitativa al resto)

1. Introducción

KNN (*k-nearest neighbors*) es un algoritmo de clasificación supervisada. Para clasificar un punto nuevo, calcula su distancia a todos los puntos de entrenamiento, toma los k más cercanos y le asigna la etiqueta que más se repite entre ellos.

En este proyecto paralelizamos el código secuencial `knn_digits_sec.py` [2], que clasifica las imágenes del conjunto *load_digits* de scikit-learn [4]. El conjunto tiene $n = 1797$ imágenes de 8×8 píxeles, es decir $d = 64$ características, y 10 clases. El código separa los datos con `train_test_split` (20 % para prueba, `random_state=42`), lo que da $n_{tr} = 1437$ puntos de entrenamiento y $n_{te} = 360$ de prueba, y usa $k = 3$. Su precisión (*accuracy*) es 0,9833, que tomamos como referencia.

El trabajo más costoso es calcular las $n_{tr} \cdot n_{te}$ distancias, y cada una se puede calcular de forma independiente de las demás. Por eso es un problema naturalmente paralelizable [5].

Solución planteada. Usamos el paradigma de memoria distribuida (paso de mensajes), en Python con mpi4py [3]. El proceso 0 (maestro) tiene los datos al inicio. Siguiendo el DAG del enunciado [1] (Figura 1):

- se envían los datos de prueba a todos los procesos (`comm.bcast`);
- se reparten los datos de entrenamiento en bloques (`comm.scatter`);
- cada proceso calcula las distancias de todos los puntos de prueba a su bloque, las ordena y guarda sus k vecinos más cercanos por punto de prueba;
- el maestro recolecta esos candidatos (`comm.gather`), elige los k más cercanos de todos y aplica la votación por mayoría.

Si un punto es uno de los k vecinos más cercanos entre todos los datos de entrenamiento, también lo es dentro de su bloque: de lo contrario, ese bloque tendría al menos k puntos más cercanos, y el punto no podría ser vecino global. Por ello, al reunir los candidatos locales y seleccionar otra vez los k de menor distancia, se recuperan los mismos vecinos del algoritmo secuencial, salvo empates de distancia.

Para preparar este informe usamos un asistente de IA [10]; en la Bibliografía describimos para qué lo usamos y en qué partes influyó.

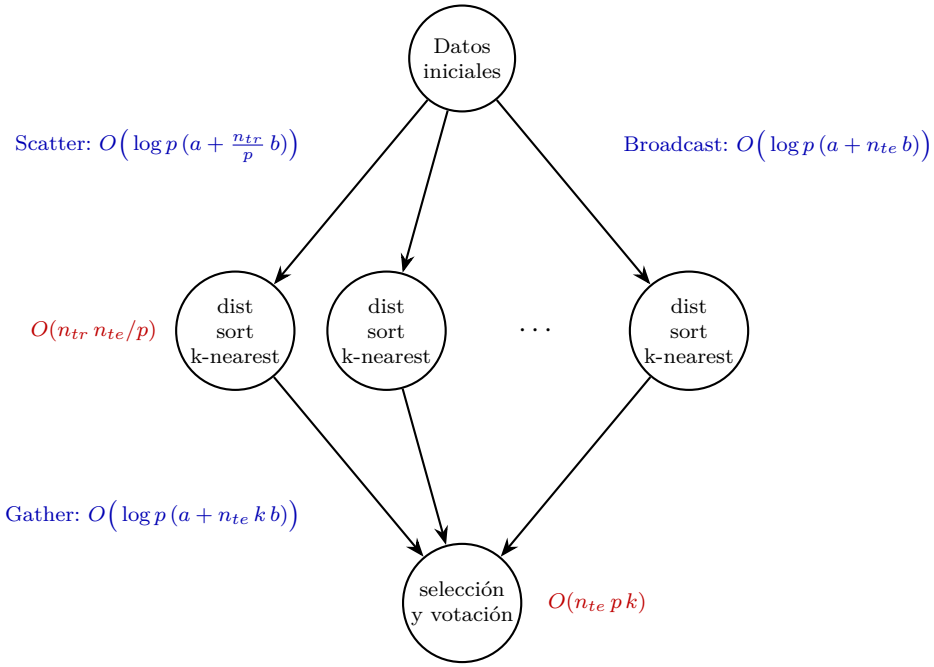


Figura 1: DAG del KNN en paralelo, adaptado de [1] con los costos de `scatter` y `gather` corregidos a $O(\log p)$ (Sección 2.5), confirmado por el profesor [5, 8]. En azul, comunicación; en rojo, cómputo. Aquí n_{te} cuenta puntos, con d constante.

2. Método

2.1. Algoritmo secuencial

Llamamos $n = n_{tr} + n_{te}$ al tamaño de los datos. La división 80/20 se mantiene fija, así que $n_{te} = c\,n$ y $n_{tr} = (1-c)\,n$ con $c \approx 0{,}2$. Con esto, el problema depende de un **solo tamaño** n, y $n_{tr}\,n_{te} = c(1-c)\,n^2 \approx 0{,}16\,n^2$. El tiempo secuencial se obtiene del código base [2] (Tabla 1).

Línea del código secuencial	**Operaciones**
`euclidean_distance(a, b)`	d restas, d productos, $d-1$ sumas y 1 raíz: $O(d)$
`distances = [... for x in X_train]`	$n_{tr} \cdot O(d)$
`np.argsort(distances)[:k]`	$O(n_{tr} \log n_{tr})$
`k_labels = [y_train[i] for i in k_indices]`	$O(k)$
`Counter(k_labels).most_common(1)`	$O(k)$
`y_pred = [knn_predict(x, ...) for x in X_test]`	lo anterior se repite n_{te} veces
`accuracy = np.mean(y_pred == y_test)`	$O(n_{te})$, una sola vez

Tabla 1: Operaciones de cada línea del código secuencial [2].

$$T_s(n) = O\big(n_{te}(n_{tr}\,d + n_{tr}\log n_{tr} + k) + n_{te}\big) = O\big(n^2(d + \log n)\big)$$

Como $d = 64 > \log_2 n \approx 10{,}8$, el cálculo de distancias domina al ordenamiento:

$$T_s(n) = O(d\,n^2)$$

2.2. Diseño con el método de Foster

La Figura 2 resume los cuatro pasos de Foster [5].

Particionamiento. Usamos descomposición del dominio sobre la data de mayor tamaño, la de entrenamiento ($n_{tr} > n_{te}$). La tarea primitiva es la distancia entre un punto de prueba y uno

de entrenamiento, lo que da $n_{tr} \cdot n_{te} = 517\,320$ tareas de costo $O(d)$ y de tamaño parecido, más de cuatro órdenes de magnitud por encima de $p = 16$ [5].

Comunicación. Cada tarea necesita sus dos puntos, y su distancia se usa luego para elegir los k vecinos de ese punto de prueba. Es decir, las tareas de una misma fila se combinan en una reducción. Contando solo las salidas, son del orden de $n_{tr}\, n_{te} \approx 5{,}2 \times 10^5$ canales hacia la selección. Esta comunicación es *overhead*: no existe en el secuencial.

Aglomeración. Agrupamos en una tarea todas las columnas de un bloque de puntos de entrenamiento. Así, cada proceso recibe solo su bloque y los datos de prueba, y envía k candidatos por punto de prueba en lugar de todas sus distancias. Contando un canal de Broadcast, uno de Scatter y uno de Gather por bloque, quedan del orden de $3p$ canales.

Mapeo. Asignamos un bloque por proceso. Como $n_{tr} = 1437$ no es múltiplo de p para $p = 2, 4, 8, 16$, los primeros n_{tr} mód p procesos reciben $\lceil n_{tr}/p \rceil$ puntos y el resto $\lfloor n_{tr}/p \rfloor$. Los bloques difieren en un punto como máximo, así que la carga queda balanceada y en el análisis usamos n_{tr}/p [5]. Con $p = 16$ son unos 48 canales.

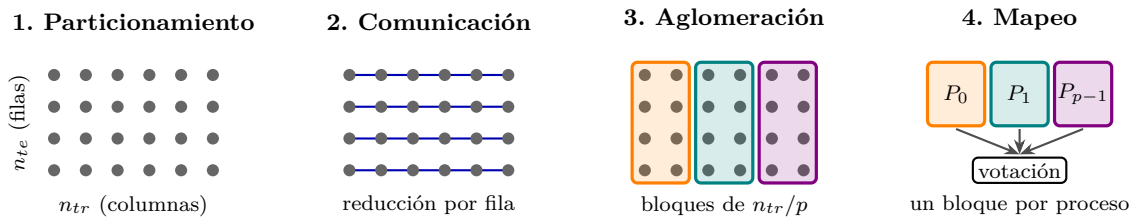


Figura 2: Método de Foster aplicado al KNN. Cada punto es una distancia entre un punto de prueba y uno de entrenamiento.

El enunciado también menciona repartir los datos de prueba. Esa alternativa (*scatter* de prueba y *broadcast* de entrenamiento) es válida y evita el paso 6 en el maestro. Sin embargo, seguimos la distribución del DAG, que reparte el conjunto de mayor tamaño.

2.3. PRAM

El Algoritmo 1 presenta el pseudocódigo con la notación PRAM del curso [5, 6]. No depende de p: los límites de los bucles se expresan solo con el tamaño del problema, y **pardo** va en el bucle sobre n_{tr}, que es la dimensión que se distribuye. El número de procesos aparece recién al ejecutar ese **pardo**, cuando cada proceso toma un bloque de valores de j y aplica la línea 6 solo a ellos; por eso el Gather reúne varias listas de candidatos por punto de prueba. Esto permite leer directamente el trabajo W y el *span* T_∞ (Sección 2.4).

Descripción de los pasos:

- **Paso 1:** el maestro envía los n_{te} datos de prueba y k a todos los procesos.
- **Paso 2:** el maestro reparte los datos de entrenamiento y sus etiquetas en bloques.
- **Paso 3:** se calculan las distancias de cada punto de prueba a los puntos de entrenamiento. Cada proceso calcula las de su bloque.
- **Paso 4:** por cada punto de prueba, cada proceso ordena las distancias de su bloque y guarda sus k vecinos locales (distancia y etiqueta).
- **Paso 5:** el maestro recolecta en un solo `Gather` los candidatos de todos los puntos de prueba, para no pagar la latencia n_{te} veces.
- **Paso 6:** por cada punto de prueba, el maestro elige los k pares de menor distancia entre todos los candidatos y aplica la votación. No se debe votar directamente con todos los candidatos: se votaría con más de k vecinos y el resultado podría diferir del secuencial.
- **Paso 7:** el maestro calcula la precisión comparando $\hat{y}$ con y_{te}.

Si lo vemos como un PRAM de memoria compartida, el algoritmo es CREW [5]: todos leen

Algoritmo 1: KNN distribuido con Broadcast/Scatter/Gather($X_{tr}, y_{tr}, X_{te}, y_{te}, k$)

Entrada: datos de entrenamiento X_{tr} ($n_{tr} \times d$) con etiquetas y_{tr}; datos de prueba X_{te} ($n_{te} \times d$) con etiquetas y_{te}; k vecinos

Salida: etiquetas predichas $\hat{y}[1, \ldots, n_{te}]$ y precisión en el proceso raíz

```
1  Broadcast(X_te, k, root = 0)                                   // paso 1
2  Scatter((X_tr, y_tr), root = 0)                                // paso 2
3  for i ← 1 a n_te do
4      for j ← 1 a n_tr pardo
5          D[j] ← sqrt(Σ_{l=1}^{d} (X_te[i,l] − X_tr[j,l])^2)     // paso 3
6      L[i] ← los k pares (D[j], y_tr[j]) de menor distancia      // paso 4
7  Gather(L, C, root = 0)                                         // paso 5
8  if es el proceso raíz then
9      for i ← 1 a n_te do
10         V ← los k pares de menor distancia en C[i]             // paso 6
11         ŷ[i] ← etiqueta más frecuente en V
12     accuracy ← (cantidad de i con ŷ[i] = y_te[i]) / n_te       // paso 7
13     return ŷ, accuracy
```

X_{te} al mismo tiempo y cada uno escribe solo en sus posiciones. En memoria distribuida, la lectura concurrente se reemplaza por el Broadcast del paso 1.

Optimización del paso 6

En el Algoritmo 1, el maestro hace solo todo el paso 6 y recibe todos los candidatos. Ambos costos crecen con p. Podemos evitarlo porque elegir los k menores es una operación asociativa: si $A \oplus B$ denota los k pares de menor distancia de $A \cup B$, entonces $(A \oplus B) \oplus C = A \oplus (B \oplus C)$. A diferencia de una suma [5], $\oplus$ no cuesta tiempo constante, sino $O(k)$ por punto de prueba; como k es constante, la cota no cambia. Así, la selección se puede hacer como una **reducción en árbol**, en la que cada nivel fusiona listas de k pares y solo reenvía k pares por punto de prueba. El Algoritmo 2 reemplaza las líneas 7 a 11 del Algoritmo 1; las líneas 12 y 13 no cambian. En mpi4py corresponde a `comm.reduce` con una operación propia, que para objetos de Python usa una reducción en árbol por defecto [3].

Algoritmo 2: Pasos 5 y 6 optimizados con una reducción en árbol

```
1  Reduce(L, C, op = ⊕, root = 0)                                 // pasos 5 y 6
2  if es el proceso raíz then
3      for i ← 1 a n_te do
4          ŷ[i] ← etiqueta más frecuente en C[i]
```

Conservamos ambas versiones: el Algoritmo 1 cumple con el `comm.gather` que pide el enunciado, y el Algoritmo 2 es la mejora que compararemos con él.

2.4. Trabajo y *span*

Al no depender de p, el pseudocódigo permite leer el trabajo W y el *span* T_∞, que es el tiempo con procesadores ilimitados, es decir, con un valor de j por procesador. El bucle sobre i es secuencial y el bucle sobre j es paralelo. Por cada i, la línea 5 cuesta $O(d)$ en el camino crítico.

Medida	Con `Gather` (Alg. 1)	Con reducción (Alg. 2)
Trabajo $W(n)$	$O(d\,n^2)$	$O(d\,n^2)$
Paso 3 en el camino crítico	$O(n_{te}\,d) = O(d\,n)$	$O(d\,n)$
Selección en el camino crítico	$O(n_{te}\,n_{tr}) = O(n^2)$	$O(n_{te}\,k \log n_{tr}) = O(k\,n \log n)$
Span $T_\infty(n)$	$O(n^2)$	$O\big(n(d + k \log n)\big)$
Paralelismo W/T_∞	$O(d)$	$O\big(d\,n/(d + k \log n)\big)$

Tabla 2: Trabajo y *span* de ambas versiones. Sin reducción, el maestro revisa todos los candidatos, por lo que su trabajo secuencial acota el paralelismo.

La Tabla 2 justifica la optimización: con `Gather`, el paso 6 del maestro queda en el camino crítico y limita el paralelismo a $O(d)$. Con la reducción, el paralelismo crece aproximadamente como $O(n)$.

Con el teorema de Brent, $T_p = O(W/p + T_\infty)$, pasamos del *span* a un tiempo con p procesos. Como Brent no incluye la comunicación, se suma aparte [5]:

$$T_p = O\Big(\frac{d\,n^2}{p} + n^2\Big) + T_{comm}, \qquad T'_p = O\Big(\frac{d\,n^2}{p} + n(d + k \log n)\Big) + T_{comm}$$

La primera cota es mayor que la de la Sección 2.5, y no es una contradicción. Brent acota la simulación del PRAM, donde el maestro revisa n_{tr} pares por punto de prueba. Al aglomerar en p bloques, cada bloque envía solo k candidatos, y el maestro revisa $p\,k$ pares. Por eso, en la Tabla 3, el paso 6 cuesta $O(n_{te}\,p\,k)$ y no $O(n_{te}\,n_{tr})$.

El bucle sobre i también es independiente [1]. Con **pardo** en i y en j, y con la reducción, el *span* bajaría a $O(d + k \log n)$. Dejamos **pardo** solo en j porque es la dimensión que se distribuye.

2.5. Métricas

Para la comunicación usamos el modelo del curso [5]:

$$T_{comm} = f(p)\,(\alpha + x\beta)$$

donde α es la latencia, β el tiempo de transferencia por unidad de dato, x la cantidad de datos y $f(p)$ depende de la topología. En el DAG, $a \equiv \alpha$ y $b \equiv \beta$.

Siguiendo la corrección del profesor [5, 8], tomamos $f(p) = \log p$ para las tres colectivas, es decir, **ninguna paga p veces la latencia ni p veces el término de datos**:

- **Broadcast:** cada proceso recibe el mensaje completo, de tamaño $x = n_{te}\,d$, con costo $\log p\,(a + n_{te}\,d\,b)$.
- **Scatter:** cada proceso recibe solo su bloque, de tamaño $x = \frac{n_{tr}}{p}\,d$, con costo $\log p\,(a + \frac{n_{tr}}{p}\,d\,b)$.
- **Gather:** el maestro recibe los k candidatos por punto de prueba de cada proceso, $x = n_{te}\,k$, con costo $\log p\,(a + n_{te}\,k\,b)$.

Un modelo más detallado describe estas colectivas como un árbol binomial de $\log_2 p$ niveles [7]: en el *scatter*, cada nivel reenvía la mitad de lo que tiene, y en el *gather* el maestro termina recibiendo físicamente $(p-1)$ bloques en vez de p. Esto puede hacer que la comunicación medida crezca algo más rápido que $\log p$, sobre todo en el *gather*; lo retomamos en la Sección 3.7 si las mediciones lo muestran.

La Tabla 3 muestra el costo de cada paso del Algoritmo 1.

Tiempo de ejecución. Sumando los pasos y usando $n_{te}, n_{tr} = O(n)$; los tres términos $a \log p$ de la Tabla 3 se absorben en uno solo, y el término de datos del *scatter*, $\frac{n}{p}\,d\,b \log p$, queda dominado

Paso	Líneas	Qué se ejecuta	Costo
1	1	Broadcast de los datos de prueba	$T_{comm_1} = O\big(\log p\,(a + n_{te}\,d\,b)\big)$
2	2	Scatter de los bloques	$T_{comm_2} = O\big(\log p\,(a + \frac{n_{tr}}{p}\,d\,b)\big)$
3 y 4	3–6	Distancias y ordenamiento	$T_{comp_1} = O\big(\frac{n_{te}\,n_{tr}}{p}(d + \log \frac{n_{tr}}{p})\big) = O\big(\frac{d\,n^2}{p}\big)$
5	7	Gather de k pares por punto	$T_{comm_3} = O\big(\log p\,(a + n_{te}\,k\,b)\big)$
6 y 7	8–13	Selección, votación y precisión	$T_{comp_2} = O(n_{te}\,p\,k)$

Tabla 3: Costo de cada paso del Algoritmo 1.

por el del *broadcast*, $n\,d\,b\log p$:

$$T_p(n,p) = O\Big(\frac{d\,n^2}{p} + k\,n\,p + a\log p + b\,n\log p\,(d+k)\Big)$$

A diferencia de la versión con el costo del *gather* sin corregir, aquí ningún término de comunicación crece como p: el único término lineal en p, $k\,n\,p$, viene del cómputo del maestro en el paso 6 (revisar $p\,k$ candidatos por punto de prueba), no de la comunicación.

Speedup. Con $S = T_s/T_p$, donde T_s es la misma rutina de distancias vectorizada, en un proceso y sin MPI. El curso pide especificar la versión secuencial considerada [5]. No usamos el código base [2] porque el *speedup* mezclaría la vectorización con el paralelismo; su tiempo se informa aparte.

$$S(n,p) = O\left(\frac{d\,n^2}{\dfrac{d\,n^2}{p} + k\,n\,p + a\log p + b\,n\log p\,(d+k)}\right)$$

Eficiencia. Con $E = S/p = T_s/(p\,T_p)$, y dividiendo entre $d\,n^2$:

$$E(n,p) = O\left(\frac{1}{1 + \dfrac{k\,p^2}{d\,n} + \dfrac{b\,(d+k)\,p\log p}{d\,n} + \dfrac{a\,p\log p}{d\,n^2}}\right) = O\left(\frac{1}{1 + \dfrac{p^2}{n}}\right)$$

En el último paso tomamos a, b, d y k como constantes y conservamos el término dominante, porque $p\log p \le p^2$. Este resultado no cambia respecto a cuando el *gather* todavía llevaba p en su término de datos: el término dominante, $k\,p^2/(d\,n)$, viene del cómputo del paso 6, no de la comunicación.

Con el paso 6 optimizado. La reducción reemplaza T_{comm_3} por $O\big(\log p\,(a + n_{te}\,k\,b)\big)$ y T_{comp_2} por $O(n_{te}\,k\log p)$, con lo que ya no queda ningún término que crezca linealmente con p:

$$T'_p(n,p) = O\Big(\frac{d\,n^2}{p} + k\,n\log p + a\log p + b\,n\,(d+k)\log p\Big), \qquad E'(n,p) = O\left(\frac{1}{1 + \dfrac{p\log p}{n}}\right)$$

Isoeficiencia. Usamos la función de *overhead* $T_0 = p\,T_p - T_s$: para mantener E constante, T_s debe crecer como T_0, y cada término de T_0 da una condición, de la que manda la más restrictiva [5]. Con a, b, d y k constantes, $T_0 = O(n\,p^2 + n\,p\log p + p\log p)$. Comparando cada término con $T_s = O(n^2)$ se obtienen $n \propto p^2$, $n \propto p\log p$ y $n \propto \sqrt{p\log p}$, así que n debe crecer al menos como p^2: $n = \Omega(p^2)$. Con la reducción, $T'_0 = O(n\,p\log p + p\log p)$ y basta $n = \Omega(p\log p)$, de modo que esa versión es más escalable. Ambas condiciones se validan con las mediciones en la Sección 3.7.

Interpretación. Las expresiones $O(\cdot)$ omiten las constantes de tiempo de cada operación. El curso toma a y b del orden de 10^{-7}–10^{-9} s como referencia [5]; los valores de nuestra máquina se miden en la Sección 3.1. Para p pequeño esperamos que el tiempo esté dominado por las distancias

y que el algoritmo sea *compute bound*. Al aumentar p, ese trabajo disminuye como $1/p$, pero los costos del maestro y de la comunicación no, y la eficiencia tiende a bajar.

FLOPs. Por la fórmula de la distancia euclidiana, cada distancia hace d restas, d productos, $d-1$ sumas y 1 raíz, es decir $3d$ operaciones. La raíz cuenta como una operación aunque sea más lenta, porque los FLOP contabilizan igual operaciones distintas [5]. En la región paralelizable:

$$F(n) = 3\,d\,n_{tr}\,n_{te}, \qquad \text{FLOPs}(p) = \frac{F(n)}{T_p}$$

También mostramos $F(n)/\max_q t_{comp}^{(q)}$, que excluye la comunicación. Para $n = 1797$ se tiene $F = 3 \cdot 64 \cdot 517\,320 \approx 9{,}93 \times 10^7$ operaciones.

Normalización. Para comparar la teoría con las mediciones, reemplazamos cada término de T_p por un coeficiente en segundos que absorbe d, k, c, a, b y el costo real de cada operación:

$$\hat{T}(n,p) = c_1\frac{n^2}{p} + c_2\,n\,p + c_3\,n\log_2 p + c_4\log_2 p$$

Los coeficientes $c_i \geq 0$ se ajustan por mínimos cuadrados con las mediciones. Para la versión con reducción, $\hat{T}'$ no lleva el término c_2. Cada coeficiente tiene un significado físico: c_1 refleja el tiempo por operación de distancia, c_2 el costo de los candidatos en el maestro, c_3 el tiempo por dato y c_4 la latencia. Por eso comparamos c_3 y c_4 con los b y a medidos con un ping-pong [5]. Además ajustamos por separado T_{comp} y T_{comm}, que se miden por separado.

Número óptimo de procesos. Derivando respecto de p e igualando a cero (en $\hat{T}$ despreciamos además los términos logarítmicos, menores que $c_2\,n\,p$):

$$p^* \approx \sqrt{\frac{c_1\,n}{c_2}} \qquad \text{y, con reducción,} \qquad p^{*\prime} \approx \frac{\ln 2\;c_1\,n^2}{c_3\,n + c_4}$$

En ambos casos son mínimos, porque la segunda derivada es positiva en ese punto: $\hat{T}''(p^*) = 2\,c_1\,n^2/p^{*3} > 0$, y $(\hat{T}')''(p^{*\prime}) = (c_3\,n + c_4)/(\ln 2\;p^{*\prime 2}) > 0$. Los compararemos con el punto en que se cruzan las curvas de cómputo y comunicación [5].

3. Resultados y análisis

[**PENDIENTE**] Esta sección se completa con las mediciones. Ningún valor experimental ha sido medido todavía.

3.1. Entorno experimental

[**PENDIENTE**] Hardware, sistema operativo, versiones de Python, mpi4py, MPI y NumPy; valores de p y n; repeticiones por configuración. Valores de a y b medidos con el ping-pong. Explicar qué reloj se usa: T_p y comunicación salen del maestro, y el cómputo es el máximo entre procesos. Para $n < 1797$ se usan submuestras estratificadas, y para $n > 1797$ se replican las imágenes con ruido entero, por lo que su precisión no es comparable con la del código base. Aclarar que en un solo nodo MPI transmite por memoria compartida, y que se fijó un hilo por proceso en NumPy.

3.2. Desarrollo en versiones beta

El código se desarrolló en tres versiones (Tabla 4). Cada una se validó contra el código base [2]: para $n = 1797$ y $p = 1, 2, 3, 4, 7, 8, 16$, las tres dieron exactamente las mismas 360 predicciones, con precisión 0,9833. Los valores $p = 3$ y $p = 7$ se incluyeron para comprobar bloques de distinto

tamaño. La versión secuencial vectorizada que se usa como T_s también dio las mismas predicciones. Los empates de distancia se resuelven por índice global, para que el resultado no dependa del reparto.

Versión	Contenido
Beta 1	`comm.bcast`, `comm.scatter` y `comm.gather` con bloques desiguales (Algoritmo 1)
Beta 2	Beta 1 con medición separada de cómputo y comunicación, y conteo de FLOP
Beta 3	Paso 6 optimizado con `comm.reduce` y una operación propia (Algoritmo 2)

Tabla 4: Versiones beta del código.

Durante el desarrollo aparecieron tres problemas que afectaban las mediciones o el costo teórico:

- **Speedup superlineal falso.** La primera medición daba $E > 1$. La causa era que `(X_b - x) ** 2` crea arreglos temporales en cada iteración [9]: en bloques mayores a unos 0,7 MB, cada distancia costaba unas 4 veces más, lo que penalizaba a los valores pequeños de p. Se corrigió reutilizando un mismo arreglo.
- **Espera de arranque.** No usamos barreras para medir, porque agregan *overhead*. Sin ellas, el maestro empezaba a contar mientras otros procesos seguían importando librerías, y esa espera aparecía como comunicación. Se corrigió ejecutando el algoritmo una vez sin medir y midiendo la segunda ejecución.
- **Costo del paso 6.** La selección se implementó ordenando los $p\,k$ candidatos, para desempatar por índice. Esto agrega un factor $\log(p\,k)$ a T_{comp_2} en la Tabla 3, que en este rango de p es pequeño.

3.3. Tiempos de ejecución, cómputo y comunicación

[PENDIENTE] Gráficas de los tres tiempos frente a p para cada n, precisión frente a n con los datos originales y granularidad $G = T_{comp}/T_{comm}$.

3.4. Comparación con la complejidad teórica

[PENDIENTE] Describir cómo se normalizó la expresión teórica (Sección 2.5), con los coeficientes c_i obtenidos y sus unidades. Superponer $\hat{T}$ y $\hat{T}'$ a las mediciones, mostrar el ajuste separado de cómputo y comunicación, e informar el error. Comparar c_3 y c_4 con b y a.

3.5. Número óptimo de procesos

[PENDIENTE] Valor de p^* estimado con los c_i, y comparación con el p de menor tiempo medido y con el cruce de las curvas de cómputo y comunicación.

3.6. *Speedup* y FLOPs

[PENDIENTE] Gráficas de *speedup* y de FLOPs frente a p, y tiempo del código base junto a T_s.

3.7. Escalabilidad

[PENDIENTE] Eficiencia para distintos n y p. Validar la isoeficiencia con los datos: medir E en pares (n, p) con n/p^2 constante y con $n/(p \log_2 p)$ constante, comprobar si E se mantiene y compararla con la E del modelo normalizado. Usar cada serie como control de la otra versión.

3.8. Mejoras a la solución

[**PENDIENTE**] Conclusiones de la comparación entre `Gather` y la reducción, y otras mejoras: `argpartition` en el paso 4 y colectivas en mayúscula con arreglos de NumPy.

Bibliografía

[1] J. Fiestas, *Computación Paralela y Distribuida: Proyecto – Laboratorio*, 2026-II. Lima, Perú: UTEC, 2026.
Impacto: define el caso 5 y el contenido del informe. De aquí tomamos el DAG (Figura 1) y la distribución de datos de la Introducción (*broadcast* de prueba, *scatter* de entrenamiento, *gather* de candidatos).

[2] `knn_digits_sec.py`, código secuencial del caso 5, adjunto al enunciado [1]. [Online]. Available: `https://drive.google.com/file/d/1wTBSh7bwT1QVvce0pFMv5tcCWsg4-HeK/view` (accessed Sep. 18, 2026).
Impacto: es el código que paralelizamos. De él obtuvimos T_s (Sección 2.1), los parámetros ($k = 3$, división 80/20, `random_state=42`) y la precisión de referencia (0,9833).

[3] L. Dalcin, *MPI for Python (mpi4py) documentation* [Online]. Available: `https://mpi4py.readthedocs.io/en/stable/` (accessed Nov. 4, 2026).
Impacto: revisamos cómo funcionan `comm.bcast`, `comm.scatter` y `comm.gather`, y que las reducciones de objetos usan un árbol por defecto (`mpi4py.rc.fast_reduce`), lo que sustenta el Algoritmo 2.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
Impacto: es la librería que usa el código base (`load_digits`, `train_test_split`). De ella salen los datos y su división, descritos en la Introducción.

[5] J. Fiestas, diapositivas y apuntes del curso CS4052: U1.1 Fundamentos de paralelismo, U1.2 Fundamentos (Foster, Flynn), U1.3 DAG, U2.1 Diseño en paralelo (*performance*), U2.2 PRAM 01, U2.3 PRAM 02, U2.4 Particionamiento, U3.2, U3.5, U6.1 y U6.2 (*performance*). Lima, Perú: UTEC, 2026.
Impacto: son la base teórica del informe: paralelización natural (Introducción), pasos de Foster y particiones balanceadas (Sección 2.2), formato del PRAM con **pardo**, pasos numerados y CREW (Sección 2.3), teorema de Brent (Sección 2.4), y $S = T_s/T_p$, $E = S/p$, isoeficiencia, FLOPs, medición de a y b y el modelo $T_{comm} = f(p)(\alpha + x\beta)$ (Sección 2.5).

[6] J. Fiestas, *CS4052 – Práctica Dirigida 02 (solución)*. Lima, Perú: UTEC, 2026.
Impacto: nos sirvió de modelo para escribir el pseudocódigo con Scatter, Broadcast y Gather (Sección 2.3) y para expresar la eficiencia en la forma $E = O(1/(1 + \dots))$ (Sección 2.5).

[7] R. Thakur, R. Rabenseifner y W. Gropp, "Optimization of Collective Communication Operations in MPICH," *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005.
Impacto: describe el modelo de árbol binomial para las colectivas, que comentamos en la Sección 2.5 como explicación física de por qué el maestro recibe $(p - 1)$ bloques en el *gather*.

[8] J. Fiestas, respuesta por correo electrónico a consulta del grupo sobre el costo de `scatter` y `gather`, 6 de noviembre de 2026 (comunicación personal).
Impacto: confirma que el costo correcto es $O(\log p)$ y no $O(p)$, lo que sustenta la Sección 2.5, la Tabla 3 y la Figura 1.

[9] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
Impacto: es la librería con la que se calculan las distancias en el código paralelo. Su manejo de arreglos temporales explicó el problema de rendimiento descrito en la Sección 3.2.

[10] Anthropic, *Claude Opus 5* y *Claude Opus 5.5* (asistentes de inteligencia artificial, usados con Claude Code), setiembre y noviembre de 2026.
Uso: en la entrega parcial, resumió y ordenó el material de clase y redactó el borrador del informe. En esta entrega, redactó en LaTeX la versión corregida según las observaciones del profesor: pseudocódigo y diagramas (Secciones 2.2 y 2.3), trabajo y *span* (Sección 2.4) y métricas (Sección 2.5). También escribió el código de las versiones beta y los scripts de validación, experimentos y gráficas, y detectó los problemas de medición de la Sección 3.2. Además, comparó el Método con las diapositivas del curso [5].
Impacto: la mayor parte de la redacción se hizo con esta herramienta, lo que ahorró tiempo. También propuso la reducción en árbol para optimizar el paso 6 (Algoritmo 2). El grupo es responsable de lo presentado.